

Parte 5 — Explicabilidade e incerteza

Tutorial completo do sciModelFlowR (versão 1.0.2)

Walter Esfrain Pereira

2026-09-19

Índice

1	Explicar sem confundir importância com causa	3
1.1	Estabilidade das explicações	6
2	Incerteza com significado declarado	8
3	Conformal: cobertura prometida, cobertura medida	10
4	Transporte: o modelo funciona fora do domínio?	14
5	Exercícios da Parte 5	16

1 Explicar sem confundir importância com causa

Um modelo pode acertar e ainda assim ser mal compreendido. A pergunta prática é dupla: **quais variáveis pesam** (importância) e **como o modelo responde** quando uma variável muda (perfis). O `sciModelFlowR` separa as duas e mantém a explicação presa ao desenho (nada de explicar no conjunto de teste).

```
campo <- ler("field_yield")
v <- verdade$field_yield
sem_out <- campo[!campo$plot_id %in% v$outlier_ids, ]

spec_p5 <- smf_experiment_spec(
  task = smf_task_spec("regression", "yield"),
  data = smf_data_spec("yield", c("nitrogen", "potassium", "water", "organic_matter",
                                   "block", "genotype"),
                        id_column = "plot_id", categorical = c("block", "genotype")),
  design = smf_design_spec(id_column = "plot_id"),
  preprocessing = smf_preprocess_spec("median"),
  resampling = smf_resampling_spec("holdout", train_prop = 0.7, seed = 21L),
  model = smf_model_spec("linear_regression", "stats"),
  metrics = list(smf_metric_spec("rmse")),
  reproducibility = smf_reproducibility_spec(21L)
)
fit_p5 <- smf_fit_experiment(spec_p5, sem_out)
teste_p5 <- sem_out[fit_p5@split@test_index, ]
```

```
imp <- smf_permutation_importance(fit_p5, teste_p5, n_repeats = 5L, seed = 9L)
tab_smf(imp[, c("feature", "importance", "sd", "n_repeats")],
       caption = "Importância por permutação: queda de RMSE quando a variável é embaralhada")
```

Tabela 1.1: Importância por permutação: queda de RMSE quando a variável é embaralhada (5 repetições).

	feature	importance	sd	n_repeats
1	nitrogen	1080,581	108,764	5
3	water	169,790	51,689	5
2	potassium	52,276	14,571	5
6	genotype	13,414	2,537	5
4	organic_matter	9,077	7,469	5
5	block	-6,600	6,523	5

```
ggplot(imp, aes(stats::reorder(feature, importance), importance)) +
  geom_col(fill = cores_smf[["primaria"]], width = .6) +
  geom_errorbar(aes(ymin = importance - sd, ymax = importance + sd), width = .2) +
  coord_flip() +
  labs(x = NULL, y = "importância (queda de RMSE, kg/ha)",
       title = "Permutation importance with SD") +
  ylim(min(0, min(imp$importance - imp$sd)), max(imp$importance + imp$sd) * 1.08)
```

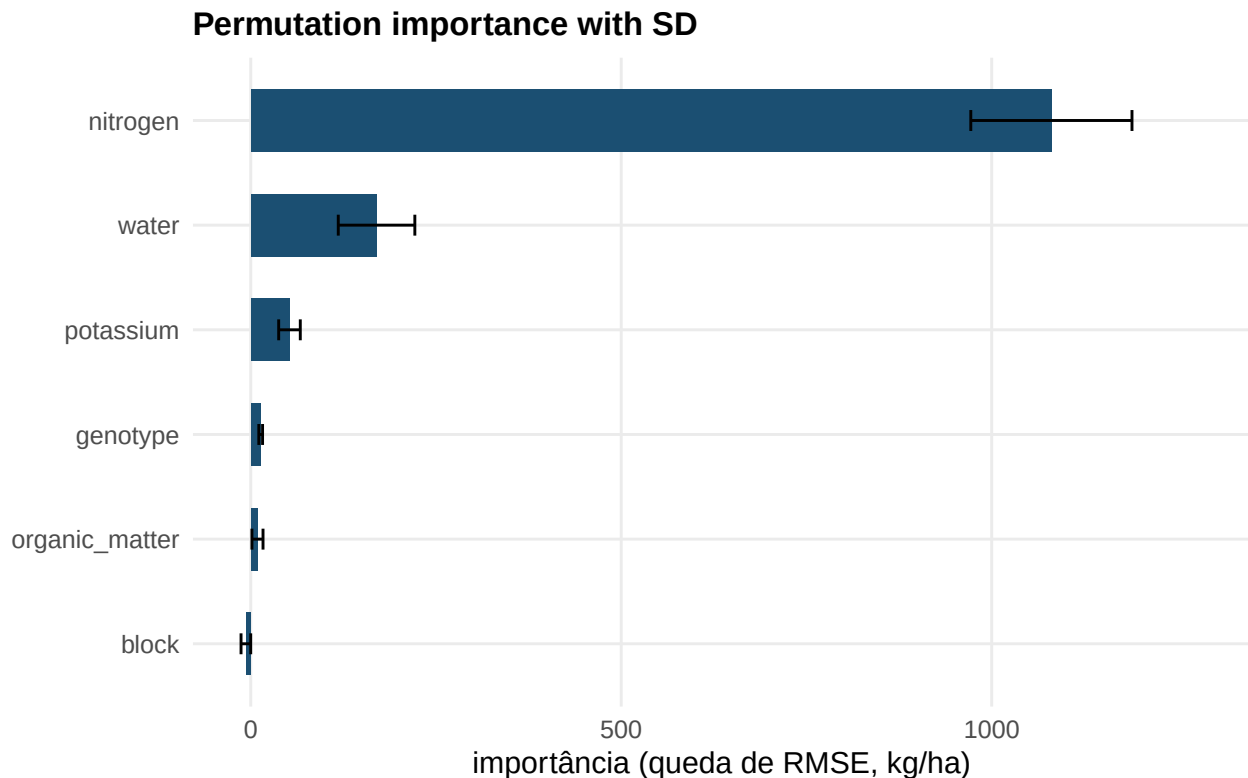


Figura 1.1: Importância com incerteza: nitrogênio domina; água e potássio vêm longe; bloco e genótipo quase não pesam porque têm poucos níveis e efeitos moderados.

Os perfis respondem “como o modelo muda com a dose”. PDP, ICE e ALE respondem à mesma pergunta por caminhos diferentes: PDP integra sobre a marginal; ICE mostra curva por observação; ALE mede efeito local em bins.

```
pdp_n <- smf_pdp(fit_p5, teste_p5, features = "nitrogen", grid_size = 12L)
ale_n <- smf_ale(fit_p5, teste_p5, features = "nitrogen", bins = 12L)
ice_n <- smf_ice(fit_p5, teste_p5, features = "nitrogen", grid_size = 12L, n = 40L, seed = 7L)
cat("PDP linhas:", nrow(pdp_n), "| ICE linhas:", nrow(ice_n), "| ALE linhas:", nrow(ale_n), "
```

PDP linhas: 12 | ICE linhas: 480 | ALE linhas: 12

```

p_pdp <- ggplot() +
  geom_line(data = ice_n, aes(value, prediction, group = row_id),
            colour = cores_smf[["claro"]], alpha = .4) +
  geom_line(data = pdp_n, aes(value, prediction), colour = cores_smf[["alerta"]],
            linewidth = 1.1) +
  labs(x = "nitrogen (kg/ha)", y = "rendimento previsto (kg/ha)",
        title = "PDP (red) over ICE (grey)")
p_ale <- ggplot(ale_n, aes(value, effect)) +
  geom_line(colour = cores_smf[["primaria"]], linewidth = 1.1) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  labs(x = "nitrogen (kg/ha)", y = "efeito local acumulado (kg/ha)",
        title = "ALE: local effect")
patchwork::wrap_plots(p_pdp, p_ale)

```

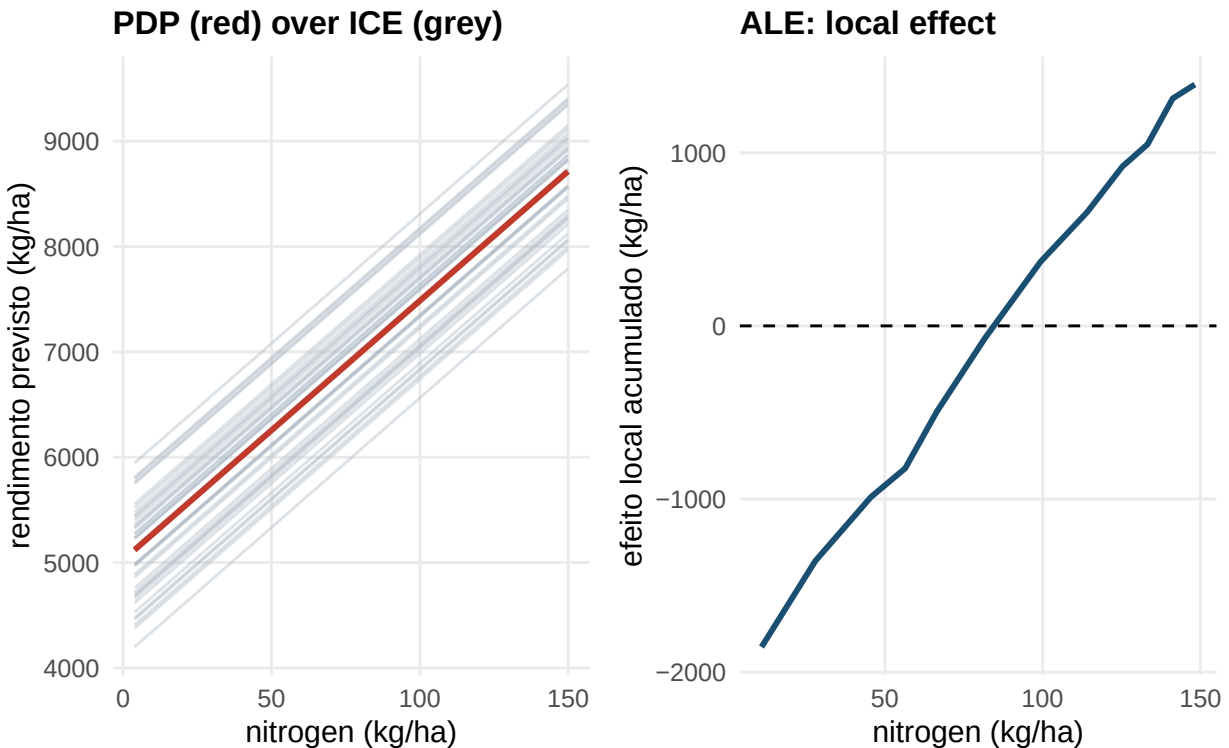


Figura 1.2: Perfis de dependência para nitrogênio: o PDP médio e as curvas ICE por parcela (esquerda) e o efeito local acumulado (direita). A inclinação recupera ~25 kg/ha por unidade.

```

loc <- smf_local_explain(fit_p5, teste_p5[1, ], sem_out, features = c("nitrogen", "water"))
tab_smf(loc, caption = "Explicação local da primeira parcela do teste: contribuição de cada v

```

Tabela 1.2: Explicação local da primeira parcela do teste: contribuição de cada variável para a diferença entre a predição e a referência.

row_id	feature	original	reference	prediction	reference_prediction	contribution
1	nitrogen	146.8	87.7	8692,292	7237,908	1454,384
1	water	574.8	510	8692,292	8500,909	191,383

```
sh <- smf_shap(fit_p5, background = sem_out[1:40, ], new_data = teste_p5[1:4, ],
               nsim = 20L, seed = 3L)
tab_smf(sh[1:4, c("plot_id", "nitrogen", "water", "organic_matter")],
        caption = "Valores SHAP para quatro parcelas (atribuição aditiva, aproximada).")
```

Tabela 1.3: Valores SHAP para quatro parcelas (atribuição aditiva, aproximada).

plot_id	nitrogen	water	organic_matter
0	1466,345	395,061	37,264
0	-1219,080	77,930	80,612
0	-616,581	35,863	-40,935
0	218,776	-172,779	-6,470

i Interpretação

As três ferramentas concordam em direção e ordem: nitrogênio domina, a curva é quase reta com inclinação próxima da verdade (25 kg/ha por unidade) e a explicação local fecha a conta (contribuição = predição - referência). Importância **não é causa**: permutar nitrogênio destrói também a estrutura correlacionada do ensaio; por isso a leitura é “o modelo usa esta variável”, não “esta variável causa o efeito”.

1.1 Estabilidade das explicações

Explicação de um único ajuste pode ser sorte. `smf_explanation_stability()` repete a explicação em folds e mede a concordância de ranking.

```
spec_cv_p5 <- smf_experiment_spec(
  task = smf_task_spec("regression", "yield"),
  data = smf_data_spec("yield", c("nitrogen", "potassium", "water", "organic_matter",
                                   "block", "genotype"),
                        id_column = "plot_id", categorical = c("block", "genotype")),
  design = smf_design_spec(id_column = "plot_id"),
  preprocessing = smf_preprocess_spec("median"),
  resampling = smf_resampling_spec("kfold", n_splits = 4L, seed = 17L),
  model = smf_model_spec("linear_regression", "stats"),
  metrics = list(smf_metric_spec("rmse")),
```

```

    reproducibility = smf_reproducibility_spec(17L)
  )
  exspec <- smf_explain_spec("permutation", n_repeats = 3L, seed = 44L)
  est_xai <- smf_explanation_stability(spec_cv_p5, sem_out, exspec, top_k = 3L)
  tab_smf(est_xai@summary, caption = "Importância média entre folds, com frequência de sinal positivo")

```

Tabela 1.4: Importância média entre folds, com frequência de sinal positivo.

feature	mean_importance	sd_importance	cv_importance	positive_fraction	n_folds
nitrogen	1146,098	90,310	0,079	1,0	4
water	180,266	37,930	0,210	1,0	4
potassium	100,424	15,385	0,153	1,0	4
block	-2,398	9,646	4,022	0,5	4
genotype	17,192	16,959	0,986	1,0	4
organic_matter	8,885	6,400	0,720	1,0	4

```

  tab_smf(est_xai@rank_stability, caption = "Estabilidade de ranking: posição média de cada variável entre folds.")

```

Tabela 1.5: Estabilidade de ranking: posição média de cada variável entre folds.

feature	mean_rank	sd_rank
nitrogen	1,0	0,000
water	2,0	0,000
potassium	3,0	0,000
block	5,5	1,000
genotype	4,5	0,577
organic_matter	5,0	0,816

! Interpretação

Com 6 variáveis e 4 folds, o ranking é estável quando **sd_rank** é pequeno e **positive_fraction** é 1. Nitrogênio aparece no topo em todos os folds; variáveis de efeito fraco trocam de posição — e isso é a informação, não um defeito: explicação instável deve ser comunicada como instável.

2 Incerteza com significado declarado

O pacote recusa tratar tipos de incerteza como sinônimos. A tabela de semânticas é o contrato: cada intervalo promete uma coisa diferente, e nenhum é intercambiável.

```
sem_u <- smf_uncertainty_semantics()
tab_smf(sem_u, caption = "Semântica canônica dos tipos de intervalo: o que cada um promete e sob quais suposições.")
```

Tabela 2.1: Semântica canônica dos tipos de intervalo: o que cada um promete e sob quais suposições.

interval_type	target	probability_statement	interchangeable
confidence	parameter or estimand	repeated-sample procedure	FALSE
credible	parameter/latent quantity	posterior probability conditional on model/prior	FALSE
prediction	future response	sampling distribution conditional on fitted model	FALSE
posterior_predictive	future response	posterior predictive probability	FALSE
bootstrap	estimator/statistic	resampling approximation	FALSE
conformal	future response or class set	marginal coverage under conformal assumptions	FALSE

```
desc_conformal <- smf_uncertainty_descriptor("conformal", "future_response", "conformal",
                                             0.90, "exchangeability")
desc_bootstrap <- smf_uncertainty_descriptor("bootstrap", "estimator", "bootstrap",
                                             0.95, "resampling approximation")
val_con <- smf_uncertainty_validate(desc_conformal)
val_boot <- smf_uncertainty_validate(desc_bootstrap)
tab_desc <- data.frame(
  rotulo = c(smf_uncertainty_label(desc_conformal), smf_uncertainty_label(desc_bootstrap)),
  valido = c(val_con$valid, val_boot$valid)
)
tab_smf(tab_desc, caption = "Rótulos de incerteza: a origem, o alvo e a suposição entram no nome.")
```

Tabela 2.2: Rótulos de incerteza: a origem, o alvo e a suposição entram no nome.

rotulo	valido
conformal 90% [conformal; target=future_response]	TRUE
bootstrap 95% [bootstrap; target=estimator]	TRUE

rotulo	valido
--------	--------

```
decomp <- smf_uncertainty_decompose(aleatoric = 0.4, epistemic = 0.6, total = 1.0,
                                   method = "ensemble", identified = TRUE)
print(decomp)
```

```
<sciModelFlowR UncertaintyDecomposition>
method: ensemble
identified: TRUE
```

```
recusa_decomp <- tryCatch(
  smf_uncertainty_decompose(aleatoric = 0.4, epistemic = 0.6, method = "unknown",
                           identified = FALSE),
  error = function(e) conditionMessage(e))
cat("decomposição sem identificação:", recusa_decomp, "\n")
```

decomposição sem identificação: Aleatoric/epistemic decomposition was requested without an iden

```
recusa_dist <- tryCatch(
  smf_uncertainty_from_distribution(
    smf_prediction_distribution("parametric",
                               list(distribution = "normal", mean = 5000, sd = 300))),
  error = function(e) conditionMessage(e))
cat("distribuição sem decomposição identificada:", recusa_dist, "\n")
```

distribuição sem decomposição identificada: This prediction distribution does not expose an ide

Armadilha

A mensagem acima é uma recusa deliberada: o pacote **não fabrica** uma decomposição aleatória/epistêmica que o objeto não sustenta. Sem um procedimento que separe as fontes (ensemble, replicações, hierarquia), o número seria ficção com aparência técnica.

3 Conformal: cobertura prometida, cobertura medida

A garantia conformal é **marginal e sob permutabilidade** — no ensaio heterocedástico, isso é exatamente o tipo de promessa que precisa ser medida. A divisão abaixo congela três papéis: treino, calibração e teste intocado.

```
sem_completo <- sem_out[stats::complete.cases(
  sem_out[, c("nitrogen", "potassium", "water", "organic_matter")]), ]
set.seed(20260919)
ordem <- sample.int(nrow(sem_completo))
i_tr <- ordem[1:90]; i_cal <- ordem[91:120]; i_te <- ordem[121:149]
f_con <- stats::lm(yield ~ nitrogen + potassium + water + organic_matter,
  data = sem_completo[i_tr, ])
p_cal <- unname(stats::predict(f_con, sem_completo[i_cal, ]))
p_te <- unname(stats::predict(f_con, sem_completo[i_te, ]))
con <- smf_conformal_fit(truth = sem_completo$yield[i_cal], prediction = p_cal,
  spec = smf_conformal_spec("split", level = 0.90),
  design = smf_design_spec(id_column = "plot_id"),
  calibration_ids = sem_completo$plot_id[i_cal],
  final_test_ids = sem_completo$plot_id[i_te])
inter <- smf_conformal_predict(con, prediction = p_te)
cob <- smf_conformal_coverage(sem_completo$yield[i_te], inter)
tab_smf(cob, caption = "Cobertura empírica do intervalo conformal de 90% no teste intocado.")
```

Tabela 3.1: Cobertura empírica do intervalo conformal de 90% no teste intocado.

n	coverage	mean_width	median_width
29	0,897	837,043	837,043

```
diag_con <- smf_conformal_diagnose(con, truth = sem_completo$yield[i_te], prediction = inter)
tab_smf(diag_con$summary, caption = "Diagnóstico conformal: cobertura contra o alvo, com tolerância declarada.")
```

Tabela 3.2: Diagnóstico conformal: cobertura contra o alvo, com tolerância declarada.

n	coverage	mean_width	median_width
29	0,897	837,043	837,043

```
tab_smf(data.frame(alvo = diag_con$target, observado = diag_con$observed,
                    erro_monte_carlo = diag_con$monte_carlo_se,
                    tolerancia = diag_con$tolerance),
        caption = "Alvo, cobertura observada e margem de erro amostral da cobertura.")
```

Tabela 3.3: Alvo, cobertura observada e margem de erro amostral da cobertura.

alvo	observado	erro_monte_carlo	tolerancia
0,9	0,897	0,056	0,111

```
ord <- order(p_te)
df_con <- data.frame(parcela = seq_along(p_te), pred = p_te[ord],
                     lo = inter$.pred_lower[ord], hi = inter$.pred_upper[ord],
                     obs = sem_completo$yield[i_te][ord])
ggplot(df_con, aes(parcela, pred)) +
  geom_ribbon(aes(ymin = lo, ymax = hi), fill = cores_smf[["claro"]], alpha = .5) +
  geom_point(aes(y = obs), size = 1.6, colour = cores_smf[["alerta"]]) +
  geom_line(colour = cores_smf[["primaria"]]) +
  labs(x = "parcela do teste (ordenada pela predição)", y = "rendimento (kg/ha)",
       title = "Split conformal intervals at 90%",
       subtitle = paste0("Cobertura medida: ", fmt(100 * cob$coverage, 1),
                          "% | largura média: ", fmt(cob$mean_width, 0), " kg/ha"))
```

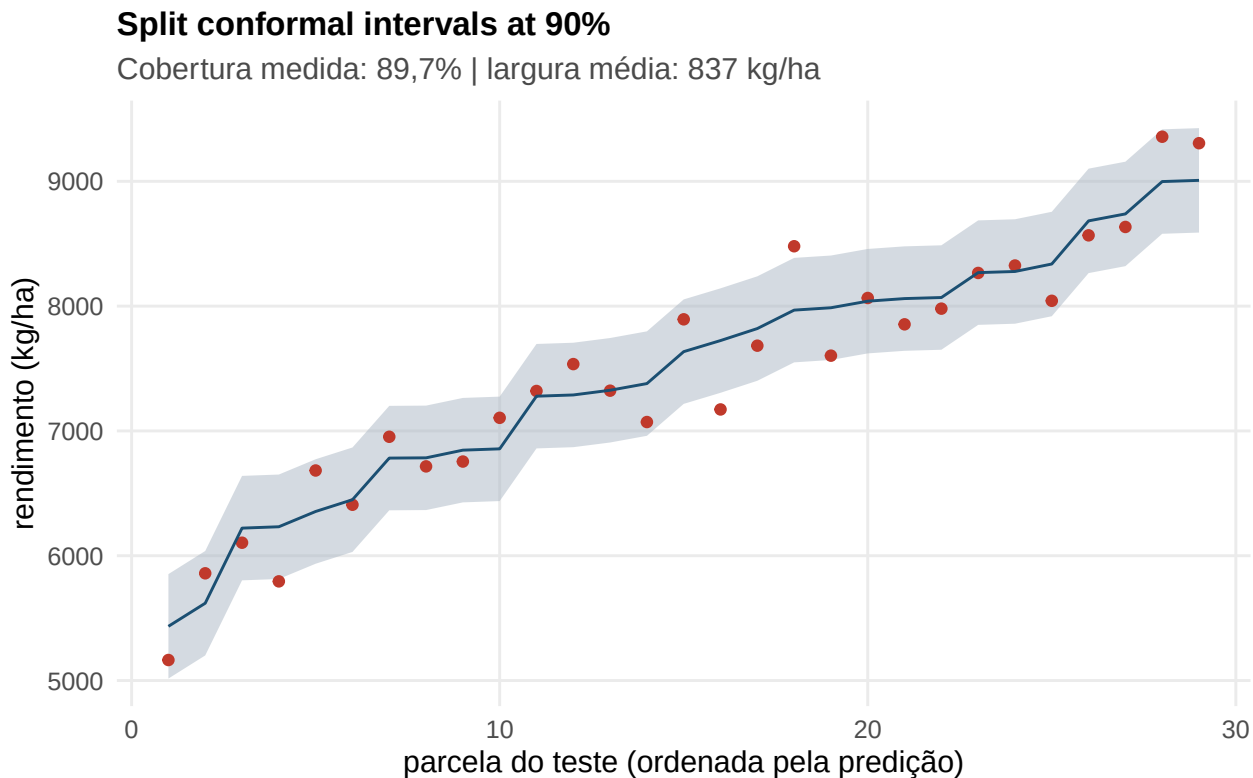


Figura 3.1: Intervalos conformais ordenados pela predição: a largura é constante (split conformal), e a cobertura é contada no teste, não presumida.

Na classificação, o alvo não é um intervalo, e sim um **conjunto** de rótulos com cobertura garantida (APS). O tamanho do conjunto é a medida de incerteza.

```
doenca <- ler("disease_binary")
doenca$disease <- factor(doenca$disease, levels = c(0, 1))
set.seed(20260919)
od <- sample.int(nrow(doenca))
tr_c <- od[1:250]; cal_c <- od[251:350]; te_c <- od[351:400]
glm_c <- stats::glm(disease ~ x1 + x2 + x3 + x4, data = doenca[tr_c, ],
  family = stats::binomial())
P_cal <- cbind(`0` = 1 - stats::predict(glm_c, doenca[cal_c, ], type = "response"),
  `1` = stats::predict(glm_c, doenca[cal_c, ], type = "response"))
P_te_c <- cbind(`0` = 1 - stats::predict(glm_c, doenca[te_c, ], type = "response"),
  `1` = stats::predict(glm_c, doenca[te_c, ], type = "response"))
aps <- smf_conformal_fit(doenca$disease[cal_c], probabilities = P_cal,
  spec = smf_conformal_spec("aps", level = 0.90))
conj <- smf_conformal_predict(aps, probabilities = P_te_c)
cob_cls <- smf_conformal_coverage(doenca$disease[te_c], conj)
tab_smf(cob_cls, caption = "Conjuntos conformais APS: cobertura e tamanho médio do conjunto r
```

Tabela 3.4: Conjuntos conformais APS: cobertura e tamanho médio do conjunto no teste.

n	coverage	mean_set_size	median_set_size
50	1	2	2

i Interpretação

A cobertura de 90% é medida, não presumida: com 29 parcelas de teste, a cobertura ficou em 89,7% e a largura média em 837 kg/ha — constante, porque o split conformal usa um único quantil dos escores. Na classificação, a cobertura é 100,0% com conjunto médio de 2,00 rótulos: quanto mais incerto o caso, maior o conjunto.

4 Transporte: o modelo funciona fora do domínio?

O bloco B4 tem efeito próprio e é candidato natural a “domínio externo”. A validação externa usa o modelo **já ajustado**, sem refit e sem recalibração.

```
sem_dom <- sem_out
sem_dom$domain <- ifelse(sem_dom$block == "B4", "external", "development")
spec_dev <- smf_experiment_spec(
  task = smf_task_spec("regression", "yield"),
  data = smf_data_spec("yield", c("nitrogen", "potassium", "water", "organic_matter"),
    id_column = "plot_id"),
  design = smf_design_spec(id_column = "plot_id"),
  preprocessing = smf_preprocess_spec("median"),
  resampling = smf_resampling_spec("holdout", train_prop = 0.8, seed = 31L),
  model = smf_model_spec("linear_regression", "stats"),
  metrics = list(smf_metric_spec("rmse"), smf_metric_spec("mae")),
  reproducibility = smf_reproducibility_spec(31L)
)
dev <- sem_dom[sem_dom$domain == "development", ]
ext <- sem_dom[sem_dom$domain == "external", ]
fit_dev <- smf_fit_experiment(spec_dev, dev)
val_ext <- smf_external_validate(fit_dev, ext, domain_name = "external")
cmp_ext <- smf_compare_validation_domains(fit_dev, val_ext)
tab_smf(cmp_ext$comparison, caption = "Métricas internas e externas lado a lado: o mesmo modelo")
```

Tabela 4.1: Métricas internas e externas lado a lado: o mesmo modelo, dois domínios.

metric	internal	external	delta	direction
rmse	366,263	416,144	49,881	minimize
mae	258,757	279,990	21,234	minimize

```
df_ext <- data.frame(
  dominio = c("interno (holdout)", "externo (bloco B4)"),
  rmse = c(cmp_ext$internal$value[cmp_ext$internal$metric == "rmse"][1],
    cmp_ext$external$value[cmp_ext$external$metric == "rmse"][1])
)
ggplot(df_ext, aes(dominio, rmse, fill = dominio)) +
  geom_col(width = .5, show.legend = FALSE) +
  geom_text(aes(label = fmt(rmse, 0)), vjust = -0.4) +
```

```
scale_fill_manual(values = c(cores_smf[["primaria"]], cores_smf[["alerta"]])) +  
labs(x = NULL, y = "RMSE (kg/ha)",  
     title = "Internal vs external validation") +  
ylim(0, max(df_ext$rmse) * 1.12)
```

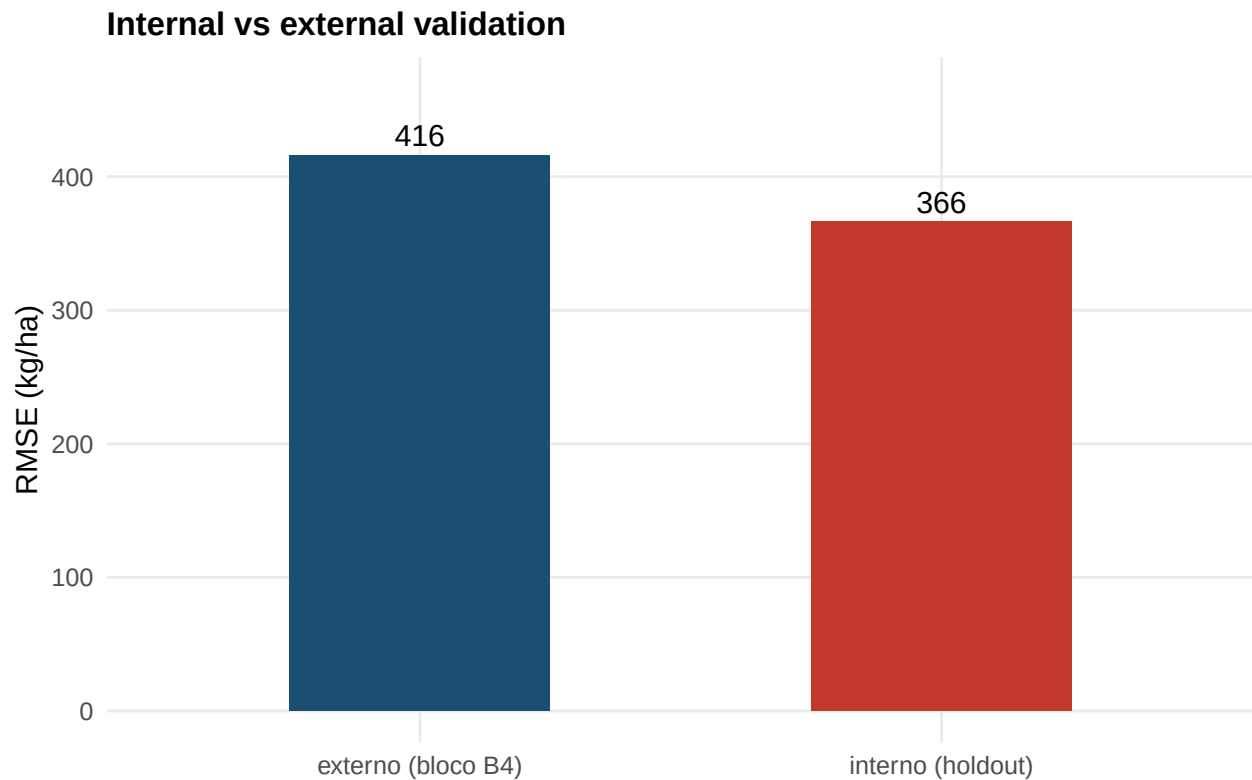


Figura 4.1: Degradação no transporte: o bloco B4 tem efeito +60 kg/ha e o modelo ajustado no desenvolvimento não o conhece — a diferença aparece na validação externa.

! Interpretação

O RMSE externo é 416 contra 366 interno — a diferença é o preço de transportar um modelo para um bloco que ele não viu. Nenhum refit foi feito: o pacote mede o transporte do artefato congelado, que é o que acontece na prática.

5 Exercícios da Parte 5

Exercício 5.1 — Importância contra causalidade

A importância do nitrogênio é enorme, mas água e potássio também entram na verdade simulada. Proponha um experimento (não computacional) que separaria efeito causal de associação neste ensaio.

Exercício 5.2 — PDP contra ALE

Compare a inclinação do PDP com a do ALE para **nitrogen**. As duas coincidem com a verdade (25)? Em que situação elas divergiriam?

Exercício 5.3 — Conformal e heterocedasticidade

Meça a cobertura conformal **por faixa de nitrogênio** (doses baixas contra altas). A cobertura marginal de 90% se mantém em cada faixa? O que isso diz sobre a garantia marginal?

Exercício 5.4 — Tamanho do conjunto como incerteza

No APS, ordene os casos pelo tamanho do conjunto. O tamanho acompanha a probabilidade prevista da classe majoritária? Construa uma figura.

Soluções

5.1. Randomização: doses de nitrogênio sorteadas independentemente das demais variáveis (como na simulação). Em dados observacionais, a associação não identifica o efeito; seria preciso um desenho causal explícito.

5.2. PDP e ALE coincidem aqui porque o modelo é linear e as variáveis são independentes. Com interações fortes ou preditores correlacionados, o PDP integra sobre combinações improváveis e o ALE localiza o efeito — as curvas divergem.

5.3. A cobertura marginal de 90% não implica 90% em cada faixa: com heterocedasticidade, doses altas tendem a ficar abaixo e doses baixas acima. É por isso que a Parte 6 apresenta o conformal **local** (CQR) e a leitura por faixa.

5.4. Sim: o conjunto cresce quando a probabilidade máxima é baixa, ou seja, quando o modelo está inseguro. A figura esperada é uma relação negativa entre probabilidade da classe prevista e tamanho do conjunto.